

Autonomous Navigation of Waypoints over Virtual Terrain using SLAM and ROS

Manimegali R
drrm@psgitech.ac.in

Srinitish D
duraisrinitish@gmail.com

Department of Computer Science and Engineering
PSG Institute of Technology and Applied Research
Coimbatore, India

Abstract—This work addresses the design and development of a terrestrial rover capable of navigating through multiple waypoints autonomously. This work employs SLAM algorithms and data from 2D Lidar sensors which can collect data from 360 degrees. ROS middleware suite framework provides the required hardware abstraction required to integrate various hardware and software components for communication. Once the waypoints are initialized through a mission plan with their respective priorities, the robot creates a provisional path to the waypoints with the highest priority. The shortest route is created based on a modified version of the A* algorithm which provides a more optimized travel plan for the robot based on obstacle probability as an additional cost factor, than the traditional A* algorithm for path planning. The proposed methodology dynamically changes its path when its vision is updated. If a waypoint is inaccessible at a particular point in travel, a special priority queue stores the waypoints and ensures that the robot loops back periodically until they are accessed. It also creates a map throughout its traversal which can be used for further navigation. It has also been identified that the global planner causes 70% of the computational complexity. The method and tools used for implementation and the results are discussed. The proposed approach initializes the robot's pose/position on the map, implementing SLAM algorithms for real-time mapping. It facilitates autonomous navigation of the vehicle through the different waypoints. Obstacle detection, avoidance and path planning.

Keywords—*Simultaneous Localization and Mapping, Robot Operating System, waypoints, 2D Lidar sensor*

I. INTRODUCTION

Autonomous navigation has witnessed rapid advancements in recent years, driven by improvements in computing power, sensor technology, and open-source robotics frameworks. One of the most influential developments in this field is the Robot Operating System (ROS), an open-source middleware suite that simplifies the integration of hardware and software components. Originally introduced in 2007 by Willow Garage, ROS has undergone several iterations, leading to significant improvements in modularity, communication efficiency, and simulation capabilities. ROS Noetic, released in 2020, is the final and Long-Term Support (LTS) release of ROS1. It introduced stability improvements, expanded package support, and better compatibility with modern Ubuntu distributions. Unlike its predecessors, ROS Noetic focused on maintaining

the longevity of ROS1 while encouraging a gradual transition to ROS2, which provides enhanced real-time capabilities and security features but does not have the same centralized setup as that of ROS 1 making it harder to perform the same functions with less computational overhead. Given these advancements, ROS Noetic remains a preferred choice for projects that require robust, well-supported tools without migrating to ROS2.

Recently there has been an increased interest in the possible use of autonomous terrestrial vehicles for lengthy or hazardous tasks such as surveillance, observation or patrolling without the need for human intervention. This has hugely been due to the advancements in computer performance and sensor technologies. For such terrestrial rovers to operate safely and accurately, systems such as path planning, localization, obstacle avoidance and mapping are crucial. These systems need to be integrated and optimized to ensure that the system is capable of working in real-time. This paper discusses the idea of modelling and creating a rover capable of identifying its surrounding environment to navigate through the checkpoints autonomously. This also involves traversing through temporary and permanent obstacles. Autonomous terrestrial vehicles are increasingly used in tasks requiring precision and adaptability, such as surveillance, patrolling, and environmental monitoring. This work addresses the critical challenges of path planning, localization, and obstacle avoidance, integrating these functionalities into a unified system. Real-time decision-making and dynamic map updates are achieved through Simultaneous Localization and Mapping (SLAM) techniques, enhancing the rover's ability to navigate complex environments autonomously.

Initial systems relied on pre-mapped environments and simple obstacle avoidance using ultrasonic sensors. SLAM models enabled robots to map and localize simultaneously. But early SLAM techniques heavily relied on probabilistic models, later Rao-Blackwellized particle filters helped improve grid-based mapping leading to more effective algorithms. The rise of the Robot Operating System (ROS) has provided a standardized framework to work with different hardware and software components enabling easier integration of different techniques. The development of such a system poses several issues majorly due to real-time

localization, planning and mapping. When there are constantly moving obstacles or the environment is dynamically changing requires continuous adaptive planning. The 2D Lidar Sensors are prone to noise interference which affects the accuracy of the map generated. The balance between the resolution and real-time performance is critical due to heavy computations in every phase. The localization drift needs to be taken into account and calibrated periodically to prevent the accumulation of errors. The path re-planning should be done effectively to prevent any deadlocks to ensure that there is no significant increase in the traversal times

II. LITERATURE REVIEW

Autonomous navigation and mapping have been widely studied, focusing on SLAM techniques and sensor integration. Malla et al. have explored object-level mapping using RFID to enhance indoor localization, emphasizing sensor fusion for navigation tasks [1]. This addressed the limitations of traditional SLAM in environments where obstacles and landmarks lack distinct features for reliable mapping. Although RFID is not directly used, the idea of enhancing navigation through multi-sensor data is relevant. However, this reliance on RFID tags limits the applicability of this approach to predefined environments where RFID infrastructure is available. Thale et al. have demonstrated SLAM implementation using the TurtleBot platform with ROS, [2] highlighting the gmapping package for real-time mapping. The work in [2] emphasized parameter tuning and computational efficiency, insights that guided the system's implementation in this study. The TurtleBot successfully mapped its environment and demonstrated real-time navigation with reduced localization drift. The paper highlights the computational challenges and tuning complexities involved in implementing SLAM on low-power robotic platforms. Similarly, Ghazal et al. have used Gazebo to simulate autonomous navigation with ROS, underscoring the importance of virtual environments in testing and validating navigation algorithms [3]. This addressed the challenge of validating autonomous navigation strategies in simulation before deploying them in real-world environments. Thus it has helped to demonstrate the reliability of Gazebo as a testbed for navigation algorithm validation, showing improvements in obstacle avoidance and waypoint traversal. While simulations provide valuable insights, real-world sensor noise and environmental uncertainties were not accounted for, making real-world performance unpredictable. In complex environments, Ben Roummane and Daoui applied ROS-based SLAM for hospital navigation, emphasizing dynamic obstacle handling and adaptive path planning, which parallels the local planning strategies used in this system [4]. This paper has thus the challenge of enabling autonomous navigation in environments with frequent human movement and dynamically changing obstacles. The system successfully navigated through hospital corridors, demonstrating adaptability to dynamic obstacles and improved patient transport efficiency. Traditional SLAM methods have suffered from poor accuracy and high computational demands when they are applied to large-scale environments. Grisetti et al. laid the foundation for many SLAM methods through Rao-Blackwellized particle filters, forming the basis for the gmapping algorithm employed in this project [5]. The method has significantly enhanced SLAM accuracy while also reducing the number of particles needed thus reducing the

computational overhead. However, the method has been shown to be lacking in environments with dynamic obstacles. With the development of various SLAM approaches, there has been a need to identify optimal solutions for different setups based on the trade-offs in terms of accuracy, computational efficiency, and real-time performance. Varanasi et al. compared SLAM algorithms such as GMapping, Hector SLAM, and Cartographer, highlighting trade-offs between accuracy and computational efficiency. Their findings influenced the choice of gmapping for its balance of performance and resource use in this system [6]. GMapping was found to offer a good balance between performance and resource usage, while Cartographer provided the best accuracy but required higher computational power. However, this study has been based on a controlled environment setup, making it difficult to generalize findings to large-scale outdoor environments. Kanishka. MS et al. laid the foundation of a parallelization algorithm for autonomous vehicles [7]. The paper addresses the computational challenges of computer vision-based autonomous navigation, specifically focusing on real-time obstacle detection and avoidance. A parallelization algorithm is proposed to distribute computer vision tasks across multiple processing units, improving computational efficiency. The approach demonstrated a significant reduction in processing time, enabling real-time obstacle detection and avoidance. Fazil has worked to develop ROS autonomous SLAM using the RRT Algorithm [8]. This work addresses the challenge of autonomous exploration and mapping in unknown environments, aiming to improve the efficiency of SLAM processes in such scenarios. This ROS package integrates the Rapidly-exploring Random Tree (RRT) algorithm for autonomous exploration. This implementation has allowed the robot to create accurate maps of unknown environments without human intervention. Dhayanithi & Mohanraj have evaluated SLAM algorithms for a Mecanum Wheeled Robot [9]. The study investigates the performance of various SLAM algorithms in enabling autonomous navigation for a mecanum-wheeled robot, particularly focusing on navigation through environments with both static and dynamic obstacles. The research has focused on implementing multiple SLAM algorithms within the ROS framework to evaluate their effectiveness in mapping and navigation tasks. The study provided insights into the strengths and weaknesses of different SLAM algorithms when applied to mecanum-wheeled robots. It also highlighted the importance of selecting the optimal SLAM algorithm based on specific environmental conditions and robot configurations. B. Liu et al. have evaluated navigation and mapping algorithms for Mobile robots based on ROS [10]. The study investigates the performance of various navigation and mapping algorithms by comparing the mapping plan and navigation period of different mapping and navigation algorithms such as Gmapping and Cartographer algorithms and Dijkstra, A * algorithm and Dynamic Window Approach algorithms respectively. It represented the importance of choosing the optimal mapping plan and navigation plan for effective traversal. Megalingam et al. have evaluated and verified the performances of ROS and SLAM based gmapping and navigation [11]. The paper has also found the time taken for traversal with and without dynamic obstacles to understand the change of the shortest path to the destination. These studies collectively inform the design and integration of SLAM, path planning, and navigation strategies in this work.

III. PROPOSED SLAM AND ROS BASED NAVIGATION SYSTEM FOR TERRESTRIAL ROBOTS

The implementation of the autonomous navigation system consists of several essential steps, including setting up the environment, creating a publisher-subscriber model, and integrating the sensor data to perform path planning and obstacle avoidance. This Navigation system is made up of both hardware and software setups. The hardware setup includes a terrestrial rover which consists of a Differential Drive Basin for locomotion and a 2D Lidar Sensor for scanning the environment.

The software stack is made up of, Robot Operating System, which is a collection of software frameworks and packages that help in abstracting and integrating various hardware and software components such as ROS Noetic support ROS1. The Gazebo application is an open-source 3D robotics simulator, it can render 3D objects under the rules of physics as configured in the ODE physics engine and supports sensor and actuator simulation. Rviz is a 3D visualization tool used to envision the sensor reading such as the laser in real time. This helps to analyze the situation and understand the different testing requirements. Figure 1 shows the created environment in plan view, it also shows the robot in the created environment at the bottom of the map inside the enclosed space.

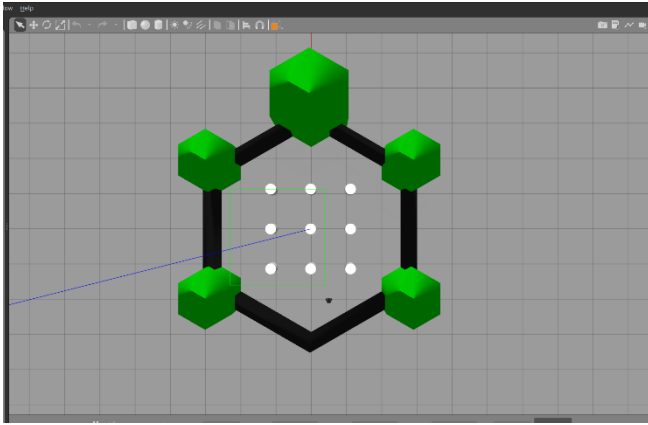


Figure. 1. Virtual Environment with obstacles indicated by white circles and Robot indicated by the black dot

The block diagram in Figure 2 explains the overall architecture and flow of the Autonomous Navigation System throughout all the allocated and matched waypoints achievable over the terrain. the system makes use of several onboard sensors, including a Lidar sensor, an Inertial Measurement Unit (IMU), and wheel encoders to collect real-time environmental data. The IMU provides orientation and motion data, the Wheel Encoders track velocity and odometry, and the Lidar sensor takes a two-dimensional laser scan of the environment. The Simultaneous Localization and Mapping (SLAM) module receives these sensor inputs, processes them, and uses the Gmapping method and Adaptive Monte Carlo Localization (AMCL) to estimate the robot's position while creating and updating an occupancy grid map of the terrain. The Waypoint Selection Module then identifies, tracks, and manages waypoints by continuously monitoring their status, ensuring that all reachable waypoints are attained while dynamically handling obstacles. The module indicates a waypoint that is momentarily unavailable

because of obstructions and periodically reassesses its status to look for new routes. Path Planning Module uses a Local Planner, like the Dynamic Window Approach (DWA), for adaptive path changes and real-time obstacle avoidance, and a Global Planner, as a modified version of the A* algorithm, for optimal route development to establish the most efficient trajectory for navigation. The Control and Execution Module then translates the planned trajectory into motor commands for movement. To validate and monitor the entire navigation process, visualization tools such as RViz and Gazebo are used to obtain real-time sensor readings and movements in the 3D environment by the robot.

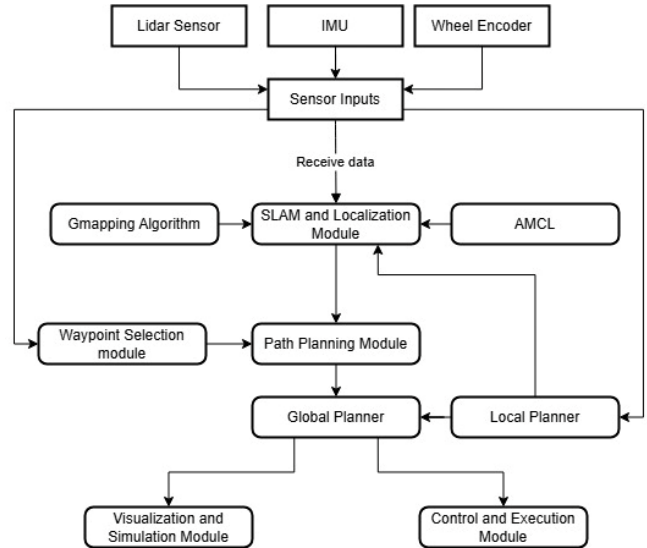


Figure.2. Block Diagram of the proposed Navigation System for Autonomous Robots

IV. NAVIGATION IMPLEMENTATION

For the autonomous navigation system, several different algorithms are consolidated into a single system, the Gmapping Package is used for SLAM and the creation of the 2D occupancy grid from the laser scan data. The Move Base algorithms are used for path planning and navigation of the robot.

Algorithm 1 Gmapping Algorithm for creating 2D Occupancy Grid

function gmapping_node():

```
{
    initialize ROS node 'slam_gmapping'
    load parameters (sensor range, map resolution, etc.)
    subscribe to 'scan' topic for LiDAR data
    subscribe to 'tf' topic for robot pose and odometry data
    while node is active:
    {
        receive LiDAR scan data and robot pose
        apply particle filter to estimate robot's pose
        update 2D occupancy grid map with new data
        publish updated map to 'map' topic
    }
}
```

end function

Adaptive Monte Carlo Localization (AMCL) algorithms are used for particle filter-based localization on a known map. This adaptive method is used to make the probability distribution as accurate as possible while also maintaining computational efficiency. The global planner uses a modified A* algorithm to ensure computational efficiency and path optimality. The heuristic is dynamically adjusted based on the obstacle density in the environment. The local planner employs a dynamic window approach, prioritizing collision-free trajectories. Algorithm 1 contains the gmapping algorithm which is a SLAM algorithm that uses the point cloud data from the lidar sensors to localize the robot and generate the map accurately. Algorithm 2 contains the modified version of the A* algorithm [12]. This algorithm is used by the global planner to plan a path to the next waypoint from the current position and it is a type of heuristic search enabling optimized path planning.

Algorithm 2 Modified A* Algorithm for Global Planner

```
function modified_a_star(start, goal, global_costmap):
{
    create open_list and closed_list
    add start node to open_list with cost 0
    while open_list is not empty:
    {
        current_node = node with lowest (g + h) in open_list
        if current_node == goal:
            return reconstructed path from start to goal
        remove current_node from open_list
        add current_node to closed_list
        for each neighbour of current_node:
            if neighbor in closed_list or is_obstacle(neighbor,
            global_costmap):
                continue
            tentative_g_score = g_score[current_node] +
            cost(current_node, neighbor)
            if neighbor not in open_list or tentative_g_score <
            g_score[neighbor]:
            {
                set parent of neighbour to current_node
                set g_score[neighbor] = tentative_g_score
                set h_score[neighbor] = heuristic(neighbor, goal)
                add neighbor to open_list
            }
        }
    }
    return "path not found"
end function

function heuristic(node, goal):
    return euclidean distance between node and goal

function cost(node1, node2):
    return distance(node1, node2) +
    penalty_for_obstacles(node2)

function is_obstacle(node, costmap):
    return costmap[node] > threshold
}
```

V. WAYPOINT TRAVERSAL

The Waypoint selection module is responsible for the identification and tracking of the waypoints assigned and reached. Thus it performs waypoint identification, where it recognizes and assigns waypoint priority based on predefined mission objectives. waypoint tracking is responsible for continuously monitoring the robot's progress in reaching each assigned waypoint, ensuring successful navigation while avoiding obstacles. When an assigned waypoint is inaccessible, it is tracked by using a modified version of the priority queue data structure. Based on the queue structure, the robot is made to periodically loop back to the previously unattainable waypoints to track their status and check for any new opening to the previously unattained waypoints. The frequency of looping back depends on the priority of the missed waypoints and the cost to loop back based on the distance to the waypoint. Adaptive Re-Evaluation performs the task of periodically re-evaluating previously unreachable waypoints, looping back to check for any changes in the environment, such as cleared obstacles or alternative paths. If a waypoint remains inaccessible after the traversal to all other waypoints, it waits for a period of time proportional to the priority of the unattained waypoint before ending the mission with the queue containing the unreachable waypoints. Algorithm 3 contains the Waypoint Traversal Algorithm which contains the modules for Waypoint Navigation and Adaptive Re-Evaluation. By integrating these functionalities, the Waypoint Selection Module contains a robust navigation system that ensures that the robot can effectively adapt to changing environments and maximize its success in reaching all assigned waypoints.

Algorithm 3 Successive Waypoint Traversal Algorithm

```
function waypoint_selection_module(start, waypoints,
global_map):
{
    initialize priority_queue with waypoints based on priority
    and distance
    initialize visited_waypoints as an empty set
    initialize unreachable_waypoints as an empty priority
    queue

    while priority_queue is not empty:
    {
        current_waypoint = dequeue highest priority waypoint
        from priority_queue

        if is_reachable(current_waypoint, global_map):
        {
            navigate_to_waypoint(current_waypoint)
            add current_waypoint to visited_waypoints
        }
        else
            add current_waypoint to unreachable_waypoints with
            priority
    }

    while unreachable_waypoints is not empty:
    {
        current_waypoint = dequeue highest priority waypoint
        from unreachable_waypoints
    }
```

```

if has_environment_changed(global_map):
{
    if is_reachable(current_waypoint, global_map):
    {
        navigate_to_waypoint(current_waypoint)
        add current_waypoint to visited_waypoints
    }
    else
    {
        reinsert current_waypoint into
        unreachable_waypoints with updated priority
    }
}
wait_for_next_evaluation(current_waypoint)
}

return visited_waypoints, unreachable_waypoints
}

function is_reachable(waypoint, global_map):
return path_exists(start_position, waypoint, global_map)

function navigate_to_waypoint(waypoint):
execute_pathPlanning_Planner(cur_position,
dest_position)

function has_environment_changed(global_map):
return check_for_new_openings(global_map)

function wait_for_next_evaluation(waypoint):
sleep(time_based_on_priority(waypoint))

```

VI. SLAM CALIBRATION AND LOCALIZATION

The implementation of the system involves SLAM implementation and Move Base implementation. The SLAM implementation involves simultaneously mapping and localizing the mobile robot on the generated map. This is one of the biggest problems faced when creating an autonomous robot. Here a Lidar i.e. Light Detection and Ranging, based SLAM implementation is being used where primarily a laser sensor is used. The laser outputs a 2D point cloud data which helps to make high-precision distance measurements which help to construct the map. By sequentially registering the point cloud data the system can localize itself within the created map by using the calculated distance. Map Creation involves both the generation of the newly visible environment and the updating of the old map dynamically when a new obstacle is identified. There are multiple ROS packages available to implement such a system, but in this paper, the gmapping package is used. The slam_gmapping package provides a node from which we can generate a 2-D occupancy grid. This is developed using the pose and laser data from the terrestrial vehicle. When the 2D point cloud data is received from the laser it creates the occupancy grid based on the probability of obstacles in the different parts of the map which are currently visible to the robot. Localization involves identifying the pose of the mobile robot relative to its surroundings. Here the AMCL is used for localization of the robot within the generated map. It employs a particle filter

which is a technique that uses a set of particles to find the actual probability distribution which is used to find the robot's pose within the generated map. AMCL is implemented to maintain computational efficiency. Figure 3 shows the visualization in Rviz of the robot with its initial sensor readings and map. This only contains a partially formed map due to the limited field of view of the robot. This map will be updated once the robot starts traversing through the environment.

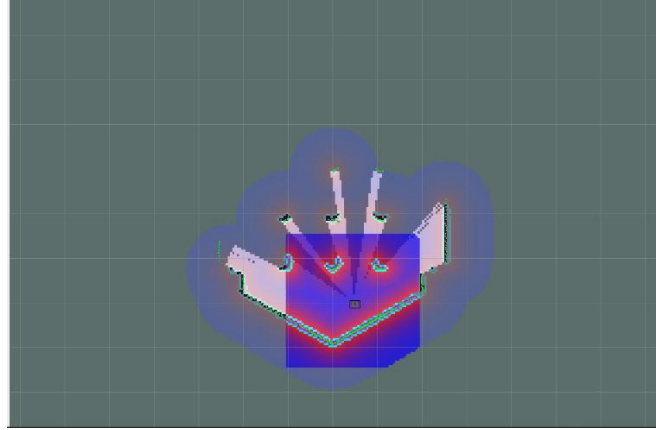


Figure. 3. Pose Initialized Sensor Data in Rviz

VII. REAL-TIME PLANNER FOR AUTONOMOUS ROBOTS

For the planning of the path to be taken, the move_base package is used. It provides the node required to link the local and global planner. The global planner uses the data from the global_costmap stored in the map_server to construct the path between the two waypoints finding the shortest path using a modified version of the A* algorithm. This is then dynamically modified by the local planner which takes the sensor data makes real-time changes and sends the movement messages to the base_controller to move the mobile robot. The waypoint coordinates are initialized before the start of traversal. The path planner then marks and traverses these points one by one. If a waypoint is unreachable at a certain point during the traversal, the remaining waypoints are attempted to be reached first. The unreachable waypoints are then enqueued to the priority queue data structure and inputted to the path planner until all the waypoints are reached. If the time limit threshold is exceeded, the robot ends the mission.

The global Planner generates an optimal path to the next waypoint from the current robot position using the pre-built map. This uses a modified version of the A* algorithm to find the shortest path without any obstacles. It only focuses on long-term path optimizations and disregards the local dynamic obstacles. In contrast, the local Planner adjusts the global path in real-time based on dynamic obstacles and sensor data to avoid collisions caused by dynamic obstacles. It uses a dynamic window approach (DWA). Its only purpose is to smoothen the navigation by avoiding obstacles within the limited field of view. Figure 4a) shows the global path plan which only takes the initial sensor data to plan the path. A black line in the Rviz tool represents the path. This plan is only made once per travel and hence made to be more optimized and has a time complexity of $O(bd)$ where b is the

branching factor and d is the depth of search, this costs the most time in the move base system. The complexity depends on the resolution and higher resolution results in a finer grid but also increases the complexity, therefore a balance should be identified. Figure 4b) displays the update caused by the local planner due to the obstacle present in the path planned by the global planner which was not identified before. This causes a slight diversion from the main path and rejoins it once a suitable temporary alternative is found. This is marked by the red line in the local planner.

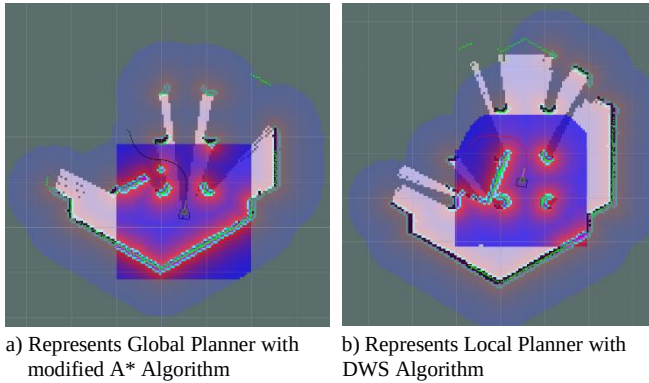


Figure. 4. Visualizes the real-time construction of the path by the global and local planner.

VIII. CONCLUSION

In this paper, we have discussed the design and development of an autonomous terrestrial navigation system capable of traversing through multiple waypoints without any human intervention. SLAM algorithms like gmapping are used to simultaneously map the region around the robot and localize the robot. This used in successive steps is used to traverse through multiple waypoints successfully. These predefined waypoints are achieved as in the mission plan and inaccessible waypoints are looped back periodically. It has also been calculated that 70% of the computational time is used for the calculation of the global path by the global planner when using the traditional form of the A* algorithm but the modified algorithm provides the benefit, hence the optimization should focus on the depth of resolution required by the global planner. Thus effective traversal between the different waypoints was successfully performed by adequately initializing the robot's position. SLAM algorithms were effectively implemented to allow accurate localization and mapping by using properly calibrated sensor data and mitigating errors caused due to drifts. The navigation between the different waypoints was also completed by appropriately avoiding both static and dynamic obstacles in the environment. Future work will focus on optimizing the SLAM and path-planning algorithms to enhance computational efficiency and reduce latency. Implementing advanced techniques such as graph-based SLAM or incorporating 3D

sensors could improve the system's mapping accuracy and obstacle detection capabilities. Additionally, transitioning from simulation to a physical hardware setup will involve addressing challenges like sensor calibration, power management, and real-world environmental conditions. Further exploration into multi-robot systems and collaborative mapping could extend the system's applicability to larger and more complex environments.

REFERENCES

- [1] Malla, H., Purushothaman, P., Rajan, S.V. and Balasubramanian, V., 2014, November. Object level mapping of an indoor environment using RFID. In 2014 Ubiquitous Positioning Indoor Navigation and Location Based Service (UPINLBS) (pp. 203-212). IEEE.
- [2] Thale, Sumegh & Prabhu, Mihir & Thakur, Pranjali & Kadam, Pratik. (2020). ROS based SLAM implementation for Autonomous navigation using Turtlebot. ITM Web of Conferences. 32. 01011. 10.1051/itmconf/20203201011.
- [3] Ghazal, Mohammad & Al-ghadhanfari, Murtadha & Waisi, Najwan & Ghazal, Mohammed. (2024). Simulation of autonomous navigation of turtlebot robot system based on robot operating system. Bulletin of Electrical Engineering and Informatics. 13. 1238-1244. 10.11591/eei.v13i2.6419.
- [4] Ben Roummane, Hamza & Cherki, Daoui. (2023). Localization and Navigation of ROS-Based Autonomous Robot in Hospital Environment. 10.1007/978-3-031-37872-0_12.
- [5] Grisetti, G., Stachniss, C. and Burgard, W., 2007. Improved techniques for grid mapping with rao-blackwellized particle filters. IEEE transactions on Robotics, 23(1), pp.34-46.
- [6] S. D. Varanasi, M. Tammana and R. K. Megalingam, "Robotic Navigation Unveiled: A Comprehensive Study of GMapping, Hector Slam, and Cartographer," 2024 3rd International Conference for Innovation in Technology (INOCON), Bangalore, India, 2024, pp. 1-6, doi: 10.1109/INOCON60754.2024.10511508.
- [7] K. MS, R. Manimegalai and N. M. SK, "Parallelization Algorithm of Computer Vision for Autonomous Vehicles," 2023 2nd International Conference on Vision Towards Emerging Trends in Communication and Networking Technologies (ViTECoN), Vellore, India, 2023, pp. 1-6, doi: 10.1109/ViTECoN58111.2023.10157031.
- [8] Fazil, M. (2020). ROS Autonomous SLAM Using RRT Algorithm. GitHub Repository.
- [9] Dhayanithi, P. V., & Mohanraj, T. (2023). ROS-Based Evaluation of SLAM Algorithms and Autonomous Navigation for a Mecanum Wheeled Robot. In Intelligent Control, Robotics, and Automation (pp. 13-24). Springer.
- [10] B. Liu, Z. Guan, B. Li, G. Wen and Y. Zhao, "Research on SLAM Algorithm and Navigation of Mobile Robot Based on ROS," 2021 IEEE International Conference on Mechatronics and Automation (ICMA), Takamatsu, Japan, 2021, pp. 119-124, doi: 10.1109/ICMA52036.2021.9512584.
- [11] Megalingam, Rajesh Kannan & Chinta, Ravi & Sreekanth, Sarath & Raj, Akhil. (2019). ROS based Autonomous Indoor Navigation Simulation Using SLAM Algorithm.
- [12] Yang, Liwei & Li, Ping & Wang, Tao & Tian, Jiya & Chen, Chuangye & Tan, Jie & Wang, Zijian. (2024). Multi-area Collision-free Path Planning and Efficient Task Scheduling Optimization for Autonomous Agricultural Robots. 10.21203/rs.3.rs-4038385/v1.